Input Encoders for Multi-Channel Signal Transformers: An Empirical Audit of Subspace Geometry

Ossi Lehtinen*

Ocon Oy

Abstract

Transformers consuming multi-channel scalar signals must embed C simultaneous values into one d_{model}-dimensional vector per time step. We compare eight input encoders—spanning per-channel linear projections, an orthogonality regulariser, a nonlinear MLP stem, block-partitioned concatenation, channel-independent and channel-as-token architectures, and a projected positional encoding—on a synthetic benchmark where channel identity is informative and on ETTh1 as a real-data check. At convergence the standard per-channel linear projection (`nn.Linear(C,` $d_{\texttt{model}}$`)`) matches every alternative we test to within seed noise on the synthetic benchmark. A variant that projects the sinusoidal positional encoding through a learned linear layer (LINEAR-PPE) shows a small (∼0.05 NLL) advantage at C=4 on five seeds—suggestive but at the edge of what our sample resolves, and with the underlying mechanism (positional–channel orthogonalisation vs. positional subspace compression) not pinned down by the present data. On ETTh1 the per-channel-W_k family again ties at the top, but the channel-as-token baseline (CAT), which underperforms on the synthetic task, here posts the lowest mean NLL—with overlapping confidence intervals and indistinguishable accuracies, so we read it as the synthetic-benchmark gap closing rather than a clean win. A gram-matrix analysis shows the task loss drives channel projections to near-orthogonality without explicit regularisation, and learned norms are systematically larger for the outcome-relevant channels. Code: https://github.com/OssiLehtinen/FDM-encoding.

1 Introduction

Transformers applied to multivariate numerical signals—multi-sensor telemetry, financial factor stacks, biomedical waveforms—face a design choice at the input layer: how to embed C simultaneous scalar channels into one d_{model}-dimensional vector per time step.

Prior art. The additive embedding recipe inherited from NLP (1) projects each input to a vector and sums. When the inputs are multiple channels rather than one token-plus-position pair, this comes in two forms: a *shared* scalar projection W with per-channel additive embeddings, or a *per-channel* projection W_k—the latter being mathematically a single `nn.Linear(C,` $d_{\texttt{model}}$`)`. Many multivariate time-series transformers use a linear input stem (2, 3), though details vary. Alternative architectures treat each channel as a separate token (iTransformer (5)), run independent backbones per channel (PatchTST (4)), or combine both (Crossformer (6)). Orthogonality penalties on weight matrices have been used for feature decorrelation in deep networks (7).

*ossi@ocon.fi

Open questions. Despite the variety of approaches, several basic questions remain unanswered in the literature: (i) Does a shared-scalar projection with a small channel embedding actually work, or does it have a hard ceiling? (ii) Is block partitioning, an orthogonality penalty, or nonlinearity needed, or does a bare linear layer suffice? (iii) Does the task loss itself produce channel separation without explicit enforcement? (iv) Does the positional encoding interact with the channel encoding, and can this interaction be improved?

Our contribution. We compare eight encoders on a controlled synthetic benchmark designed to make channel identity informative, with supporting experiments on the public ETTh1 dataset (2). We find that: (i) the per-channel linear projection matches every alternative at convergence; (ii) a shared-scalar baseline has a sharp, capacity-independent ceiling; (iii) the task loss drives the learned W_k to near-orthogonal subspaces and scales their norms larger for outcome-relevant channels; and (iv) projecting the sinusoidal positional encoding through a learned linear layer (LINEAR-PPE) shows a small improvement at C=4 that is at the edge of what five seeds can resolve, with a mechanism (orthogonalisation vs. compression of the positional subspace) we cannot pin down from the present data.

2 Method

2.1 Model architecture

All encoders share a common backbone: a small causal transformer with d_{model}=64, 4 attention heads, 3 layers, feed-forward width d_{ff}=256, dropout 0.1, pre-LayerNorm, and a linear categorical head mapping to K=32 logits. The *only* component that varies across experiments is the input encoder.

2.2 Encoders

We study eight encoders. In the definitions below, $v_k(t)$ is the scalar value of channel k at time t, $\mathbf{p}(t)$ is a fixed sinusoidal positional encoding (1), and all learned parameters are randomly initialised.

sum (ablation baseline). Shared scalar projection $W \in \mathbb{R}^{d_{\text{model}}}$, learned per-channel additive embedding e_k:

$$\mathbf{h}(t) = \sum_k \big(W\, v_k(t) + e_k\big) + \mathbf{p}(t).$$

Total encoder capacity is $\mathcal{O}(d_{\text{model}})$ regardless of C. We use this to isolate what goes wrong when the input projection shares weights across channels.

linear. Per-channel projection W_k and bias b_k, summed:

$$\mathbf{h}(t) = \sum_k \big(W_k\, v_k(t) + b_k\big) + \mathbf{p}(t).$$

Equivalently $\mathbf{h}(t) = W\,\mathbf{v}(t) + \mathbf{b} + \mathbf{p}(t)$ with $W \in \mathbb{R}^{d_{\text{model}} \times C}$: a single `nn.Linear(C, d_model)`.

sum-ortho. Same as LINEAR with an auxiliary loss $L_{\text{ortho}} = \lambda \sum_{i \neq j} (W_i \cdot W_j)^2/2$ (λ=10^{-2}). This variant isolates the regulariser's contribution.

linear-ppe (projected positional encoding). Same channel encoding as LINEAR, but the sinusoidal position passes through a learned linear layer:

$$\mathbf{h}(t) = \sum_k \big(W_k\, v_k(t) + b_k\big) + W_{\text{pos}}\, \mathbf{p}(t) + \mathbf{b}_{\text{pos}},$$

with $W_{\text{pos}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$. This lets the optimiser rotate the positional subspace to be orthogonal to the channel subspace. Adds d_{model}^2 parameters (4,096 at d=64).

mlp. Two-layer MLP on the channel vector: $\mathbf{h}(t) = W_2\, \text{GELU}(W_1\, \mathbf{v}(t) + \mathbf{b}_1) + \mathbf{b}_2 + \mathbf{p}(t)$, with $W_1 \in \mathbb{R}^{d_{\text{model}} \times C}$, $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$. Tests whether nonlinearity helps ($\sim$9$\times$ more encoder parameters than LINEAR).

concat. Per-channel projection into d_{block}=d_{model}/C dims, concatenated (requires $C \mid d_{\text{model}}$): $\mathbf{h}(t) = [\mathbf{e}_0(t) \| \cdots \| \mathbf{e}_{C-1}(t)] + \mathbf{p}(t)$. Channel identity is an architectural invariant.

ci (channel-independent). PatchTST-spirit (4): shared-weight causal transformer per channel; final hidden states concatenated into the head. Compute: $\mathcal{O}(C\,B\,T^2)$.

cat (channel-as-token). iTransformer-spirit (5): each (t, k) pair is a token, sequence length $C{\cdot}T$, causal across time, bidirectional within a time step. Attention: $\mathcal{O}((CT)^2)$; skipped at C=16.

2.3 Training protocol

Identical across encoders and across both datasets unless noted.

Objective. At every position $t \in \{0, \ldots, T{-}1\}$ the model emits logits $\ell_t \in \mathbb{R}^K$ and we minimise the next-step categorical cross-entropy

$$\mathcal{L}_{\text{NLL}} = -\frac{1}{B(T{-}1)} \sum_{b=1}^{B} \sum_{t=0}^{T-2} \log \text{softmax}(\ell_t^{(b)})_{\,y_{t+1}^{(b)}},$$

where $y_t^{(b)}$ is the bin index at position t in series b. The predicted distribution at position t is supervised only by the bin at position $t+1$, and causal masking inside the transformer prevents the model from seeing y_{t+1} at prediction time. The SUM-ORTHO encoder adds the auxiliary loss $\lambda \sum_{i \neq j} (W_i \cdot W_j)^2/2$ with λ=10^{-2}; all other encoders have $\mathcal{L} = \mathcal{L}_{\text{NLL}}$.

Optimiser. AdamW with peak learning rate $3{\times}10^{-4}$, weight decay 10^{-4}, and PyTorch defaults for momentum (β_1=0.9, β_2=0.999, ϵ=10^{-8}). Gradient norm is clipped to 1.0 per step. Batch size is 32. No warm-up. Learning rate follows a cosine annealing schedule from peak to $3{\times}10^{-6}$ (1% of peak) over the full 300 epochs.

Train/val split. For the synthetic benchmark we generate n_{series}=512 length-T=160 series per seed and split with a seeded `torch.utils.data.random_split` into 461/51 ($\approx$ 90/10) train and val. For ETTh1 we follow the dataset's standard chronological partition, using the train block to fit per-variate standardisation statistics and the quantile bin edges for the target, and evaluating on the val block. Data loaders use `shuffle=True` for training, `shuffle=False` for validation, batch size 32 in both cases.

Validation cadence and model selection. Validation NLL and top-1 bin accuracy are evaluated every 20 epochs, plus at epoch 1 and epoch 300 (giving a trace of 17 points). We do not actually halt training early; instead, we report the best validation NLL observed along this trace, together with the accuracy at that same epoch ("effective early stopping"). This decouples our headline metric from slight differences in convergence epoch across encoders while still letting every run see the full 300-epoch schedule.

Seeds and reproducibility. Each configuration is run over 5 random seeds, indexed 0 through 4. Seed s sets `torch.manual_seed(s)` (controlling weight initialisation and dropout) and seeds the `torch.Generator` passed to `random_split` (controlling the train/val partition). On the synthetic benchmark, seed s also drives the `numpy` RNG that generates the input series themselves. *The same five seeds are reused for every encoder variant, every C, and every d_{model}.* This is a paired-seed design: at a fixed s, every encoder sees identical training data and the identical train/val partition, and only the model architecture (and its initial parameter draws from the shared PRNG state) differs. The comparisons in this paper are reported as unpaired mean $\pm$ std intervals for simplicity, but the paired structure is available in the saved per-run records and would yield tighter intervals (and a cleaner test of small effects like the LINEAR-PPE gap) under a paired-difference analysis. We do not enable `torch.use_deterministic_algorithms(True)`, so CUDA matmul nondeterminism contributes some additional seed-level variation between full re-runs; the reported $\pm$ std intervals capture this. All headline numbers in this paper are means $\pm$ sample standard deviations over the five seeds, computed at each run's best validation NLL epoch.

2.4 Datasets

Synthetic. N=512 time series of length T=160 with C channels. Each channel is a sum of three sinusoids with random frequencies in $[0.005, 0.08]$ cycles/sample plus AR(1) noise, standardised. The outcome is a fixed non-linear function of the first four channels with lags 3 and 7:

$$y_t = \tanh\big(s_0(t-3)\, s_1(t)\big) + 0.6 \sin\big(1.3\, s_2(t-7)\big) + 0.4\, \mathbb{1}[s_3(t) > 0]\, s_0(t),$$

binned into K=32 quantile bins. Channels $k \geq 4$ are independent distractors. The task is designed so that an encoder that mixes channels at the input layer cannot recover the interaction structure.

ETTh1. Electricity Transformer Temperature (2), hourly, 7 variates. We standardise on the train split, bin the target variate OT into 32 quantile bins, and predict the next-step bin given all 7 variates over T=160. The model uses d_{model}=56 (divisible by C=7), 7 heads, d_{ff}=224; training is otherwise identical.

2.5 Diagnostic analyses

Beyond aggregate NLL, we use three diagnostics to inspect what the per-channel-W_k encoders actually learn. Each is computed after full 300-epoch training, on five seeds unless stated otherwise.

Gram-matrix analysis. LINEAR and SUM-ORTHO differ only in whether the soft-orthogonality penalty $L_{\text{ortho}} = \lambda \sum_{i \neq j} (W_i \cdot W_j)^2/2$ is added to the loss. If both reach the same NLL, the question is whether LINEAR's W_k are geometrically similar to SUM-ORTHO's—i.e. whether the task pressure alone already produces a near-orthogonal arrangement—or whether the two encoders arrive at the same loss through different geometries.

After training we extract the per-channel projection matrix $W \in \mathbb{R}^{C \times d_{\text{model}}}$ (rows W_k) from the encoder and compute the cosine matrix $G_{ij} = (W_i \cdot W_j)/(\|W_i\|\|W_j\|)$. Two scalar summaries:

$$\overline{|\cos|} = \frac{1}{C(C-1)} \sum_{i \neq j} |G_{ij}|, \qquad \max|\cos| = \max_{i \neq j} |G_{ij}|.$$

Both go to zero iff the W_k become mutually orthogonal. We report seed-averaged means.

Encoding-space allocation. Orthogonality says channels are distinguishable; it does not say whether the model dedicates more representational capacity to outcome-relevant channels than to distractors. We measure two quantities, both at the encoder output:

(i) Norm $\|W_k\|$ for each channel. A larger norm means channel k's contribution $W_k v_k(t) + b_k$ has larger magnitude in the embedding, i.e. occupies more of the residual stream.

(ii) Per-channel variance fraction. The encoder output is a sum of per-channel contributions $W_k v_k(t) + b_k$. Treating each contribution as a random vector indexed by (t, batch), the per-channel variance is

$$\sigma_k^2 = \sum_{d=1}^{d_{\text{model}}} \text{Var}_{t,\text{batch}}\big[(W_k v_k(t) + b_k)_d\big],$$

and we report fractions $\sigma_k^2 / \sum_j \sigma_j^2$ on a held-out batch. This combines the size of W_k with the empirical variability of channel k's input distribution.

For both metrics, channels are partitioned into drivers ($k \in \{0, 1, 2, 3\}$, which enter the outcome formula) and distractors ($k \geq 4$, independent of the outcome). Reporting them this way lets us check whether the model independently discovered the driver/distractor split.

Linear-probing analysis. The encoder lays down a representation; the question is whether downstream attention and FFN layers preserve enough of it that each channel remains linearly identifiable several blocks deep. We test this by training closed-form ridge probes from a frozen hidden state back to the raw input.

Procedure: (1) train the encoder–transformer end-to-end as usual; (2) freeze it; (3) generate a fresh probe dataset (`MultiSignalDataset` with a different seed, so the model has not been trained on it); (4) for each layer of interest, pass the probe dataset through and collect hidden states $H \in \mathbb{R}^{N \times d_{\text{model}}}$ alongside the corresponding raw inputs $X \in \mathbb{R}^{N \times C}$; (5) solve the ridge regression

$$W_{\text{probe}} = (H^\top H + \lambda I)^{-1} H^\top X, \qquad \lambda = 10^{-3},$$

in closed form on a probe-train split; (6) evaluate held-out R^2 per channel on a probe-validation split: $R_k^2 = 1 - \text{SS}_{\text{res},k}/\text{SS}_{\text{tot},k}$.

We probe two layers: layer 0 (the encoder output, i.e. the input to the first transformer block) and layer 3 (the output of the third and final transformer block, before the LayerNorm and the categorical head). $R^2 = 1$ at layer 0 confirms the encoder writes the raw channel into a linearly recoverable subspace; R^2 at layer 3 reveals how much of that recoverability survives the depth of the model. CI and CAT are excluded because their hidden states have a fundamentally different shape (per-channel stream and per-token respectively), so a single $H \to X$ probe does not apply.

Table 1: Main comparison at C=4, d_{model}=64, 300 epochs, cosine LR decay, mean ± std over 5 seeds. Best val NLL (effective early stopping). Random baseline: NLL = $\ln 32 \approx 3.47$, acc =0.031.

Encoder	Enc. params	Val NLL ↓	Val acc ↑
SUM	320	3.252 ± 0.009	0.092 ± 0.003
CI	2,112	3.054 ± 0.007	0.139 ± 0.003
CAT	320	2.360 ± 0.073	0.210 ± 0.014
CONCAT	128	2.183 ± 0.030	0.240 ± 0.005
MLP	4,480	2.171 ± 0.029	0.244 ± 0.008
SUM-ORTHO	512	2.170 ± 0.016	0.245 ± 0.006
LINEAR	512	2.170 ± 0.016	0.243 ± 0.005
LINEAR-PPE	4,608	2.116 ± 0.034	0.257 ± 0.009

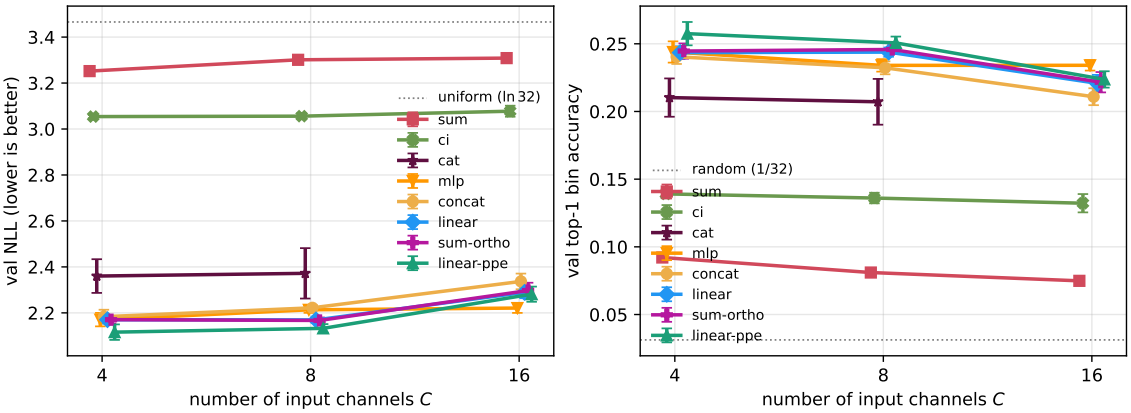

Figure 1: Scaling with input channel count C (error bars: one seed std over 5 seeds). CAT skipped at C=16 due to $\mathcal{O}((CT)^2)$ attention cost.

3 Results

3.1 Main comparison

Table 1 reports all eight encoders at C=4, d_{model}=64, 300 epochs with cosine LR decay (the same budget used throughout the paper). Four encoders sit within seed std of each other at the top: LINEAR, SUM-ORTHO, MLP, and CONCAT. LINEAR matches SUM-ORTHO to three decimal places—the orthogonality penalty contributes nothing at convergence. The MLP stem's ∼9× extra encoder parameters likewise provide no benefit. The only variant that pulls ahead is LINEAR-PPE, which projects the sinusoidal positional encoding through a learned linear layer; its ∼0.05 NLL gap exceeds seed std but is borderline at this sample size, and we return to the question of mechanism in Section 3.9 and the Discussion. CI and CAT underperform despite nominally giving each channel more capacity. The only encoder that loses decisively is SUM, which lacks per-channel projections.

3.2 Channel-count scaling

All encoders degrade as C grows (channels $k \geq 4$ are distractors). The per-channel-W_k family stays clustered at the top across $C \in \{4, 8, 16\}$. The gap to SUM widens rather than narrows. One detail worth flagging: at C=16, MLP pulls ahead of the linear family (val NLL 2.221 vs. 2.28–2.30),

Table 2: Scaling with channel count. Channels 0–3 drive the outcome; additional channels are independent distractors. 300 epochs with cosine decay, mean ± std over 5 seeds. CAT is skipped at C=16 because its $\mathcal{O}((CT)^2)$ attention cost explodes.

C	encoder	val NLL ↓	val acc ↑
4	SUM	3.252 ± 0.009	0.092 ± 0.003
	CI	3.054 ± 0.007	0.139 ± 0.003
	CAT	2.360 ± 0.073	0.210 ± 0.014
	CONCAT	2.183 ± 0.030	0.240 ± 0.005
	MLP	2.171 ± 0.029	0.244 ± 0.008
	SUM-ORTHO	2.170 ± 0.016	0.245 ± 0.006
	LINEAR	2.170 ± 0.016	0.243 ± 0.005
	LINEAR-PPE	2.116 ± 0.034	0.257 ± 0.009
8	SUM	3.301 ± 0.005	0.081 ± 0.002
	CI	3.056 ± 0.013	0.136 ± 0.004
	CAT	2.372 ± 0.110	0.207 ± 0.017
	CONCAT	2.222 ± 0.012	0.232 ± 0.005
	MLP	2.213 ± 0.013	0.234 ± 0.005
	SUM-ORTHO	2.167 ± 0.013	0.246 ± 0.003
	LINEAR	2.169 ± 0.014	0.244 ± 0.003
	LINEAR-PPE	2.133 ± 0.019	0.251 ± 0.005
16	SUM	3.309 ± 0.004	0.075 ± 0.003
	CI	3.078 ± 0.024	0.132 ± 0.007
	CONCAT	2.337 ± 0.034	0.211 ± 0.006
	SUM-ORTHO	2.297 ± 0.034	0.222 ± 0.008
	LINEAR	2.289 ± 0.026	0.221 ± 0.006
	LINEAR-PPE	2.281 ± 0.033	0.224 ± 0.006
	MLP	2.221 ± 0.021	0.234 ± 0.004

mirroring the way LINEAR-PPE pulls ahead at smaller C. With five seeds and the differences sitting at the seed-std boundary we do not read either as decisive; the point is that the top tier is narrow but its leader is not the same encoder at every C.

3.3 Model-width scaling

Table 3 varies d_{model} over $\{64, 128, 256\}$ at C=4. The SUM ceiling is sharp: 15× more parameters move NLL by less than 0.005. CONCAT's best NLL also degrades mildly as d_{model} grows from 64 to 256 ($2.18 \rightarrow 2.33$), consistent with overfitting at larger widths on this dataset.

3.4 Gram-matrix analysis: spontaneous orthogonality

LINEAR has no mechanism pushing the W_k toward orthogonality; SUM-ORTHO adds an explicit penalty. Both produce the same NLL to three decimals. Applying the off-diagonal cosine diagnostic from Section 2.5 gives:

Without any penalty the learned W_k are already near-orthogonal (mean off-diagonal cosine 0.035). The regulariser tightens this by another factor of ∼3.5 to 0.010 but does not improve

Table 3: Scaling with d_{model} at C=4. d_{ff}=4 d_{model}, heads=4 (so d_{head} grows). Mean ± std over 5 seeds, 300 epochs with cosine decay. SUM shows its structural ceiling: 15× more parameters barely move it. CONCAT's best NLL degrades mildly as d_{model} grows past 64, consistent with overfitting at larger widths on this dataset.

d_{model}	encoder	total params	val NLL ↓	val acc ↑
64	SUM	152,480	3.252 ± 0.009	0.092 ± 0.003
	CONCAT	152,288	2.183 ± 0.030	0.240 ± 0.005
128	SUM	599,840	3.253 ± 0.008	0.092 ± 0.003
	CONCAT	599,456	2.235 ± 0.014	0.235 ± 0.004
256	SUM	2,379,296	3.256 ± 0.007	0.092 ± 0.001
	CONCAT	2,378,528	2.327 ± 0.017	0.218 ± 0.006

Table 4: Learned per-channel W_k geometry (C=4, 300 epochs, mean over 5 seeds).

encoder	best val NLL	$\max_{i \neq j} \lvert\cos(W_i, W_j)\rvert$	mean $\lvert\cos\rvert$
LINEAR	2.170	0.082	0.035
SUM-ORTHO	2.170	0.022	0.010

downstream NLL. The task loss itself provides enough gradient pressure to separate the per-channel projections once they exist as free parameters.

3.5 Bias ablation: the biases play no role in the geometry

LINEAR's forward pass is $h(t) = \sum_k (W_k v_k(t) + b_k) + \mathbf{p}(t)$. Distributing the sum, the per-channel biases collapse into a single d_{model}-dimensional constant offset $B = \sum_k b_k$ that is added to every embedding regardless of input or position. Algebraically, the $C \times d_{\text{model}}$ bias parameters are an over-parameterised reparameterisation of one global offset, and they never enter the cosine computation in Table 4. To confirm experimentally that they play no role in the orthogonalisation, we retrain LINEAR with the `channel_bias` parameter zeroed and frozen (LINEAR-NOBIAS):

Table 5: Channel-bias ablation (C=4, 300 epochs, 5 seeds).

encoder	val NLL ↓	mean $\lvert\cos\rvert$	max $\lvert\cos\rvert$
LINEAR	2.169 ± 0.014	0.036	0.082
LINEAR-NOBIAS	2.177 ± 0.013	0.036	0.083

NLL is unchanged within seed std, and the Gram statistics match to three decimals. The $C \cdot d_{\text{model}} = 256$ bias parameters at this configuration are 0.17% of the full model's ∼152K parameters, so the saving is too small to drive a practical recommendation; we retain the bias in the headline definition of LINEAR to match the PyTorch `nn.Linear` default. The point of this ablation is epistemic: it confirms that the geometric argument in this paper is bias-independent.

3.6 Encoding space allocation

Beyond separating channels, does the model allocate more encoding space to channels that matter more? Using the norm and variance-fraction diagnostics from Section 2.5, at $C \in \{4, 8, 16\}$:

Table 6: Encoding space allocation for LINEAR (300 epochs, mean over 5 seeds). Channels 0–3 drive the outcome; the rest are distractors.

	$\|W_k\|$ norm		variance fraction	
	drivers (0–3)	distractors	drivers	distractors
C=8	1.21	0.55	82.9%	17.1%
C=16	1.29	0.61	60.5%	39.5%

Driver channels receive ∼2.2× larger seed-averaged norms than distractors at C=8 and ∼2.1× at C=16. The four drivers capture 83% of the embedding variance at C=8 and 61% at C=16, where the aggregate share is naturally diluted by having twelve distractors but the per-channel variance contribution remains markedly larger for drivers (∼15% each vs. ∼3% each). We do not report per-channel error bars on the norms, and the fine-grained ordering within drivers is descriptive only; the robust claim is the driver/distractor gap, which holds at both channel counts.

Why distractor norms are not zero. The driver/distractor gap is large but the distractor norms (∼0.55 at C=8) do not vanish. This is consistent with the finite-data dynamics rather than an asymptotic property of the encoder. The expected gradient on a distractor weight W_k is

$$\mathbb{E}\left[\frac{\partial \mathcal{L}}{\partial W_k}\right] = \mathbb{E}\left[\frac{\partial \mathcal{L}}{\partial \mathbf{h}(t)} \cdot v_k(t)\right],$$

which factorises to zero whenever the distractor input v_k is independent of the drivers and the outcome (true by construction in the synthetic benchmark, and approximately so for the standardised synthetic channels). With finite data, however, stochastic gradient estimates have nonzero variance whose magnitude scales as $\mathcal{O}(1/\sqrt{N_{\text{series}}})$. Under L2 weight decay (coefficient λ=10^{-4}), distractor weights reach an equilibrium whose magnitude is approximately $\sqrt{\sigma^2_{\text{noise}}/\lambda}$, i.e. they sit at a noise floor rather than being driven to zero.

Several interventions would push distractor norms down further. (i) More training data: $1/\sqrt{N}$ scaling predicts that 10× the data should shrink distractor norms by $\sim\sqrt{10} \approx 3.2$, from ∼0.55 toward ∼0.17, with driver norms essentially unchanged. We probe this prediction directly with the `geom_largen` stage of our reproducer; the result is reported alongside the standard geometry numbers in `results_paper/geom_largen.json` when that stage has been run, and may be folded into this table in a future revision. (ii) An L_1 penalty on the rows of W (or any group/row-norm sparsity regulariser) would drive distractor norms to exact zero without affecting drivers materially, at the cost of an extra hyperparameter and a deviation from the bare `nn.Linear(C,` $d_{\texttt{model}}$`)` we are auditing. (iii) Larger weight decay shrinks all norms, raising the driver/distractor ratio but at the cost of squeezing the drivers too. We do not employ any of these because the qualitative result—"the model down-weights distractors"—is already supported by the ∼2× norm gap and the variance-share split.

Table 7: Linear-probe channel recovery at C=4, seed 0, 300 epochs. Closed-form ridge probe $\mathbf{h} \mapsto \hat{x}$ from a frozen hidden state to the raw input $x \in \mathbb{R}^4$; held-out per-channel R^2. Layer 0 is the input embedding; layer 3 is the final transformer block. All per-channel-W_k encoders achieve near-perfect input recovery and preserve mean $R^2 \geq 0.84$ through three attention layers; SUM collapses to $R^2 \approx 0.24$ at every depth.

Encoder	Layer	R^2_{ch0}	R^2_{ch1}	R^2_{ch2}	R^2_{ch3}	mean
SUM	0 (input)	0.255	0.250	0.242	0.214	0.240
SUM	3 (final)	0.248	0.242	0.235	0.206	0.233
LINEAR	0 (input)	1.000	1.000	1.000	1.000	1.000
LINEAR	3 (final)	0.957	0.861	0.911	0.879	0.902
SUM-ORTHO	0 (input)	1.000	1.000	1.000	1.000	1.000
SUM-ORTHO	3 (final)	0.959	0.856	0.915	0.885	0.904
MLP	0 (input)	1.000	1.000	1.000	1.000	1.000
MLP	3 (final)	0.934	0.879	0.913	0.646	0.843
LINEAR-PPE	0 (input)	1.000	1.000	1.000	1.000	1.000
LINEAR-PPE	3 (final)	0.930	0.861	0.874	0.847	0.878
CONCAT	0 (input)	0.998	1.000	1.000	1.000	0.999
CONCAT	3 (final)	0.955	0.852	0.906	0.845	0.890

3.7 Channel identifiability via linear probing

Following the procedure in Section 2.5, all per-channel-W_k encoders achieve $R^2 \approx 1.0$ at the encoder output (layer 0) and $R^2 \geq 0.84$ at the output of the third transformer block. SUM collapses to $R^2 \approx 0.24$ at every depth—channel identity is unrecoverable from the embedding it produces.

3.8 Channel masking

Zeroing one channel at test time (Table 8), the top-tier encoders show sharp, interpretable per-channel accuracy drops that match the outcome formula. SUM is nearly flat.

3.9 Positional projection: a small additional effect

Of the encoders compared in Table 1, only LINEAR-PPE sits outside the seed-std band of the top tier. Its ∼0.05 NLL improvement over LINEAR is present at every checkpoint we record, not just at the end (2.225 vs. 2.271 NLL at epoch 100), so it is not a slow-burn late-training artefact. With only five seeds the gap exceeds either run's seed std but is at the edge of what our sample resolves; we discuss possible mechanisms (positional–channel orthogonalisation vs. positional subspace compression) in the Discussion but neither is pinned down by the present data. The channel encoding itself remains a single linear layer in either case.

3.10 Convergence and wall-clock cost

A natural worry is that more involved encoders—an MLP stem, a projected positional layer, an orthogonality regulariser—might cost extra training time even when they don't change the ceiling. From the 300-epoch traces (val NLL recorded every 20 epochs, C=4, 5 seeds) we extract two diagnostics: the epoch at which each run first reaches within 0.05 NLL of its own best, and the wall-clock seconds per epoch, normalised to LINEAR.

Table 8: Test-time channel ablation at C=4, seed 0, 300 epochs. We zero a single channel's input at inference; rows show the resulting val accuracy. Δ is accuracy drop from the unmasked baseline. The outcome formula uses ch0 in two interaction terms, ch1 in one, and ch2/ch3 once each with smaller marginal effect; the per-channel-W_k encoders reproduce this ordering cleanly, whereas SUM, CI, and CAT are flatter.

encoder	no mask	mask ch0	mask ch1	mask ch2	mask ch3
SUM	0.093	0.082	0.089	0.084	0.094
Δ acc	—	−0.011	−0.004	−0.009	+0.001
CI	0.144	0.082	0.108	0.086	0.134
Δ acc	—	−0.062	−0.036	−0.058	−0.010
CAT	0.217	0.095	0.110	0.110	0.200
Δ acc	—	−0.122	−0.107	−0.107	−0.017
CONCAT	0.241	0.096	0.113	0.103	0.202
Δ acc	—	−0.145	−0.128	−0.138	−0.039
LINEAR	0.238	0.096	0.116	0.106	0.204
Δ acc	—	−0.142	−0.122	−0.132	−0.034
SUM-ORTHO	0.239	0.096	0.117	0.104	0.205
Δ acc	—	−0.143	−0.122	−0.135	−0.034
MLP	0.240	0.098	0.111	0.106	0.209
Δ acc	—	−0.142	−0.129	−0.134	−0.031
LINEAR-PPE	0.256	0.096	0.115	0.106	0.216
Δ acc	—	−0.160	−0.141	−0.150	−0.040

The five top-tier encoders reach within 0.05 NLL of their final value at the same trace point ($\sim$150 epochs) and cost essentially the same per-epoch wall time—the transformer backbone dominates, so encoder choice within this family is free. In particular, the MLP stem's 9$\times$ extra parameters do not slow convergence. The LINEAR-PPE advantage is present at every checkpoint, not just at the end (2.225 vs. 2.271 NLL at epoch 100), so it is not a slow-burn late-training artefact.

CI and CAT pay substantial wall-clock costs from their architectures, not their input encoders: $\sim$2.3$\times$ LINEAR at C=4 for CI (scaling to $\sim$8$\times$ at C=16) and $\sim$5$\times$ at C=4 for CAT (scaling with C^2T^2 to $\sim$17$\times$ at C=8). On the synthetic benchmark CAT also converges to its worse ceiling later than the top tier reaches its ceiling.

3.11 Real-data validation: ETTh1

Table 10 mostly confirms the synthetic ordering on ETTh1: the per-channel-W_k family (LINEAR, SUM-ORTHO, CONCAT, MLP) cluster within seed std of each other, CI underperforms, and SUM fails catastrophically (NLL near $\ln 32 \approx 3.47$).

One genuine difference from the synthetic ranking: CAT, which sits in the middle of the synthetic table, here posts the lowest mean NLL (0.541). We do not want to overinterpret this. The NLL confidence interval barely separates from CONCAT/LINEAR (CAT's upper bound 0.552 versus CONCAT's lower bound 0.550), and best-bin accuracies across the top tier are indistinguishable (0.77–0.79 with seed std $\sim$0.01). Read carefully, the finding is not that channel-as-token wins on real data, but that its synthetic-benchmark disadvantage disappears. A plausible reading is that ETTh1's seven variates are all genuinely informative measurements of the same physical system, so the fine-grained cross-variate attention CAT enables can pay, whereas the synthetic benchmark's

Table 9: Convergence speed and wall-clock cost (C=4, 300 epochs, 5 seeds, single GPU). Epochs-to-target is rounded to the trace resolution of 20 epochs. Cost is per-epoch wall time normalised to LINEAR.

encoder	best NLL	epoch to NLL +0.05	relative cost
LINEAR	2.170	∼152	1.00×
SUM-ORTHO	2.170	∼156	1.04×
MLP	2.171	∼144	1.01×
CONCAT	2.183	∼152	1.00×
LINEAR-PPE	2.116	∼148	0.99×
SUM	3.252	∼20 (plateau)	1.00×
CI	3.054	∼20 (plateau)	2.3×
CAT	2.360	∼180	5.2×

Table 10: ETTh1 next-step bin prediction. 7 variates, T=160, K=32 quantile bins on the target OT (oil temperature) computed from the train split. Target is the next-step bin of OT; all 7 variates are inputs. d_{model}=56, 7 heads, 3 layers, d_{ff}=224; 300 epochs, cosine decay, 5 seeds, mean ± std of best val NLL and best val accuracy. LINEAR-PPE is included for parity with the synthetic benchmark; it does not lead here.

encoder	val NLL ↓	val acc ↑
SUM	3.633 ± 0.049	0.018 ± 0.024
CI	0.853 ± 0.022	0.674 ± 0.013
CAT	$\mathbf{0.541 \pm 0.011}$	0.787 ± 0.008
CONCAT	0.560 ± 0.010	0.777 ± 0.018
LINEAR	0.566 ± 0.019	0.789 ± 0.008
LINEAR-PPE	0.570 ± 0.015	0.773 ± 0.005
SUM-ORTHO	0.572 ± 0.022	0.787 ± 0.008
MLP	0.577 ± 0.010	0.790 ± 0.007

four drivers plus distractors does not reward that extra modelling capacity.

4 Discussion

linear is not new. Stacking per-channel projections W_k and summing is `nn.Linear(C, d_model)`—the most obvious default. Our contribution is not proposing it but auditing it: at matched parameter budget, it matches every alternative (block-partitioned, orthogonality-regularised, nonlinear, channel-independent, channel-as-token) on the synthetic benchmark, and ties at the top of the per-channel-W_k tier on ETTh1. Only the shared-projection SUM consistently loses.

What fails about the shared-scalar baseline. SUM combines a shared W with a small additive e_k. Under 15× parameter scaling its NLL moves by < 0.02. The bottleneck is not summation—LINEAR also sums—but the shared projection, which cannot represent distinct channel directions.

Orthogonality is spontaneous. LINEAR's W_k reach mean off-diagonal cosine ≈0.035 with no penalty; the regulariser tightens to ≈0.010 without changing NLL. The task loss provides the gradient pressure.

Nonlinearity is redundant. MLP (9× more encoder params) converges more slowly to the same NLL. A linear projection of the channel vector is sufficient.

Importance weighting is automatic. At C=8, driver channels get ∼2.2× larger seed-averaged $\|W_k\|$ norms and 83% of the embedding variance; at C=16 the norm ratio is ∼2.1× and per-channel variance contributions are ∼4× larger for drivers than for distractors. The driver/distractor gap is robust across two channel counts; the fine-grained ordering within drivers is descriptive only.

Projected positional encoding: small effect, unsettled mechanism. At C=4 on five seeds, LINEAR-PPE sits ∼0.05 NLL below LINEAR—a gap that exceeds either run's seed std but is borderline at this sample size, and we have not measured it at other channel counts in the converged regime. The mechanism is also not pinned down. Two readings are consistent with the data: (i) projecting position into a learned subspace orients it away from the channel directions; or (ii) it compresses position into fewer effective coordinates, freeing residual-stream capacity. Since fixed sinusoidal positions at d_{model}=64, T=160 already fill most directions, free W_k columns can in principle place themselves in the lowest-energy positional directions, so (ii) is plausible. A direct test—principal angles between span(W_k) and the effective positional basis, plus the effective rank of that basis under LINEAR vs. LINEAR-PPE—we have not run.

Architectural baselines. CI and CAT give each channel nominally more capacity at substantial wall-clock cost—∼8× LINEAR at C=16 for CI, ∼17× at C=8 for CAT. CI underperforms throughout: its bottleneck is the head combining C streams. CAT is more interesting. On the synthetic benchmark, where drivers and distractors are mixed, it sits well below the per-channel-W_k tier (2.37 vs. ∼2.17 NLL at C=4). On ETTh1, where all seven variates are informative measurements of one physical system, that gap closes and CAT posts the lowest mean NLL—though its confidence interval barely separates from the next-best encoder and best-bin accuracies in the top tier are indistinguishable. We read this as a domain-dependent split rather than a clean win: CAT's cross-variate attention appears to pay only when the channels share structure worth attending to.

No convergence-time penalty for involved encoders. MLP, SUM-ORTHO, and LINEAR-PPE reach within 0.05 NLL of their final value at the same trace point as LINEAR (∼150 epochs) and cost the same per-epoch wall time. Within the family that uses per-channel W_k on a shared backbone, encoder choice does not change either the convergence schedule or compute budget.

The residual stream carries channel identity end-to-end. Linear probes at layer 3 recover each channel to $R^2 \geq 0.84$; LayerNorm rescales but does not linearly mix coordinates. The deeper reason this works is the pre-LN transformer's residual structure: each block applies $h \leftarrow h + \text{Attn}(\text{LN}(h))$ and $h \leftarrow h + \text{FFN}(\text{LN}(h))$, so an unmodified copy of the input embedding is carried forward through every block. The per-channel projections injected at the input therefore persist into deep hidden states via the residual stream itself, not because attention or FFN sublayers learn to preserve them. Whatever near-orthogonal channel directions LINEAR establishes at the embedding layer are still linearly recoverable several blocks deep because LayerNorm only rescales them and the additive residual keeps them in the basis. This makes the geometric argument in this

paper self-consistent: per-channel-W_k encoders write the channels into a near-orthogonal subspace at layer 0, and the architecture preserves that subspace by construction.

Reading for non-numerical inputs. The mechanism we identify is geometric—distinct input streams need approximately orthogonal subspaces in d_{model} so the model can recover stream-of-origin—and is encoding-agnostic. For multi-field categorical inputs, bag-of-embeddings pooling is the discrete analog of SUM: field identity is irrecoverable unless the per-field embedding tables happen to land in disjoint subspaces, and an off-diagonal Gram penalty across embedding matrices would transfer directly. For multimodal tokens, per-modality projection heads (already standard) are the fix; the sharper claim is that they must remain *linearly separable in the residual stream*, not merely come from different distributions. The LINEAR-PPE finding—that projecting positional signals into a learned subspace beats adding them—is the most readily transferable: it applies equally to discrete sequence models where position competes with token semantics for the same coordinates. We do not test any of this; the variance-contribution diagnostic in particular does not translate cleanly to discrete inputs and would need a probe-based analog.

5 Limitations

- The synthetic benchmark deliberately makes channel identity informative. On tasks with exchangeable channels, SUM may not be penalised.
- Main sweep at d_{model}=64, $C \in \{4, 8, 16\}$; ETTh1 at d_{model}=56, C=7. The d_{model} sweep covers only SUM and CONCAT. At larger scales the overfitting dynamics may change.
- CAT is skipped at C=16 on the synthetic benchmark for compute reasons ($\mathcal{O}((CT)^2)$ attention).
- LINEAR-PPE was tested at $C \in \{4, 8, 16\}$ on the synthetic benchmark and at C=7 on ETTh1.
- Five seeds per configuration; marginal differences in the top tier may need more seeds to resolve.

6 Related work

Channel-as-token and channel-independent TSF. iTransformer (5) treats each variate as a token; PatchTST (4) runs independent backbones per channel; Crossformer (6) does both. These preserve per-channel capacity at the cost of longer sequences or per-channel compute. Our encoders share one token per time step.

Linear input projections in TSF. Many multivariate transformers use a linear stem (2, 3). We isolate this bare projection and compare it explicitly against the shared-scalar alternative and architectural baselines.

Orthogonality regularisation. Bansal et al. (7) use soft-orthogonality penalties for feature decorrelation. We apply the same idea to per-channel input projections and find it unnecessary: the task loss produces the same geometry.

7 Conclusion

On tasks where channel identity is informative, a per-channel linear projection (`nn.Linear(C,` $d_{\texttt{model}}$`)`) matches every more elaborate encoder we tested. The task loss drives the channel projections to near-orthogonality, and learned norms are larger for outcome-relevant channels. A shared-scalar projection has a capacity-independent ceiling. Nonlinearity, block partitioning, orthogonality penalties, and channel-independent architectures do not improve on the bare linear layer within seed noise. Two variants deserve a footnote rather than a full carve-out: LINEAR-PPE shows a small synthetic-benchmark advantage at C=4 that is at the edge of what five seeds resolve, and channel-as-token (CAT) posts the lowest mean NLL on ETTh1 with overlapping confidence intervals and indistinguishable accuracies—a domain-dependent narrowing of the synthetic gap, not a robust win. The practical recommendation is simple: use `nn.Linear(C,` $d_{\texttt{model}}$`)` for the channel encoding, and spend the complexity budget elsewhere unless your channels are dense, informative measurements of a single system, in which case CAT is worth the wall-clock cost.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

[2] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang. Informer: beyond efficient transformer for long sequence time-series forecasting. In *AAAI*, 2021.

[3] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long. TimesNet: temporal 2D-variation modeling for general time series analysis. In *ICLR*, 2023.

[4] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam. A time series is worth 64 words: long-term forecasting with transformers. In *ICLR*, 2023.

[5] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long. iTransformer: inverted transformers are effective for time series forecasting. In *ICLR*, 2024.

[6] Y. Zhang and J. Yan. Crossformer: transformer utilizing cross-dimension dependency for multivariate time series forecasting. In *ICLR*, 2023.

[7] N. Bansal, X. Chen, and Z. Wang. Can we gain more from orthogonality regularizations in training deep networks? In *NeurIPS*, 2018.